
Cours n° 2

RECHERCHES DANS UNE LISTE

I PARLONS LISTES ET TABLEAUX**1 LA DIFFÉRENCE ?**

Un **tableau** informatique est une suite finie de valeurs stockées dans des cases mémoires contiguës.

Le principe fondamental d'un tableau est que le $i^{\text{ème}}$ élément du tableau peut être lu et modifié en temps constant, c'est à dire indépendamment de i , en contrepartie un tableau a une taille fixe.

Une **liste** (chainée) est une suite de valeurs stockées dans des cases, l'accès au premier élément est direct et pointe vers le 2e éléments, qui pointe vers le 3e...

Le principe fondamental d'une liste est qu'il est facile d'ajouter un élément en début ou fin de liste (redimensionable) mais l'accès au $i^{\text{ème}}$ éléments demande de parcourir les autres.

Python utilise le meilleur des deux : des **tableaux redimensionnables** : on peut rapidement accéder à chaque éléments, et en ajouter ou enlever facilement.

Un petit point amusant sur les tableaux :

```
>>> t = [5,3,8]
>>> a = t
>>> a[1] = 7
>>> a
[5, 7, 8]
>>> t
[5, 7, 8]
```

Mais... mais... on ne lui a jamais demandé de modifier t !

Les noms des tableaux sont simplement des pointeurs vers une autre adresse mémoire :

```
>>> id(a),id(t)
(108546440, 108546440)
```

Il faut donc être prudent.

2 SLICING

On peut faire des extractions de tableaux : `t[a:b]` prend les valeurs de `t` entre les positions `a` inclus et `b` exclus. Notons les versions : `t[a:]` tout à partir de `a`, `t[:b]` tout jusqu'à `b` exclus.

On crée un nouveau tableau en mémoire, donc `t[:]` permet de recopier le tableau sans risquer de le modifier!

```
>>> t=[3,5,4,8,2,1]
>>> t[1:4]
[5, 4, 8]
>>> t[2:]
[4, 8, 2, 1]
>>> t[:1]
[3]
```

3 CRÉATION AVEC UN FOR

Il est possible de créer des tableaux via une définition avec un `for` :

<code>[i for i in range(11) if i%2==0]</code>	<code>[x**2 for x in range(5)]</code>	<code>[x**2 for x in range(5) if x%2==0]</code>
<code>[0, 2, 4, 6, 8, 10]</code>	<code>[0, 1, 4, 9, 16]</code>	<code>[0, 4, 16]</code>

II RECHERCHE SÉQUENTIELLES

1 RECHERCHE DE LA PRÉSENCE D'UN ÉLÉMENT

Il s'agit de chercher de manière séquentielle la présence d'une valeur dans un tableau. On compare la valeur cherchée à chaque valeur du tableau, cette méthode est appelée *méthode par balayage* ou *recherche linéaire* ou *linear search* dans la langue outre manche.

On souhaite aussi arrêter de chercher dès que l'élément est présent (pourquoi continuer?). Le fonctionnement d'un `return` rends ça facile, une fonction s'arrête quand elle tombe sur un `return`.

Les fonctions présentées prennent donc en argument un tableau (mais cela fonctionne pour une chaîne de caractère, et plus..) et l'élément à chercher. Il doit renvoyer un booléen selon que l'élément est ou non dans le tableau.

DÉFINITION 1 (*Coût d'un algorithme*)

Le **coût** d'un algorithme est déterminé par le nombre d'opérations élémentaires (affectations, comparaison, opérations de bases) exécutés par l'algorithme.

REMARQUE. On a la notion de coût moyen, de coût dans le pire cas, etc. On verra cela au fur et à mesure.

Dans notre cas, pour le pire des cas, celui où l'élément n'est pas dans le tableau, on parcourt le tableau en entier. Si la taille du tableau est n , on a donc n comparaisons à faire dans le pire des cas. On parle de *coût linéaire* et on le note $O(n)$.

Et donc la fonction :

```
1 def recherche(tab, val):
2     for elt in tab :
3         if elt == val:
4             return True
5     return False
```

REMARQUE. En python on peut aussi faire directement `val in tab` pour avoir la réponse.

2 RECHERCHE DE POSITION

On commence par chercher la position de la première occurrence de l'élément dans le tableau.

Il faut faire un choix si l'élément n'est pas dans le tableau. Ici on choisit de renvoyer `None`. C'est une variable spéciale qui est : rien.

```
1 def recherchepos(tab, val):
2     for indice in range(len(tab)):
3         if tab[indice] == val :
4             return indice
5     return None
```

3 RECHERCHE D'EXTREMA

On se place dans le cas où un tableau contient des valeurs que l'on peut comparer (des nombres, mais pas seulement). Une première fonction qui renvoie le maximum d'un tableau :

```
1 def maximum(tab):
2     maxi = tab[0]
3     for val in tab :
4         if val > maxi :
5             maxi = val
6     return maxi
```

Et une deuxième qui renvoie la position du maximum :

```
1 def posmaximum(tab):
2     pos = 0
3     for indice in range(len(tab)) :
4         if tab[indice] > tab[pos] :
5             pos = indice
6     return pos
```

III RECHERCHES SÉQUENTIELLES 2

Dans cette partie on travaille sur des tableaux de tableaux de nombres.

EXEMPLE 1. Donner les valeurs des appels suivants, avec `tab2d = [ [1,0,5,4], [2,7,6], [3,7,4,8,6] ]`

- | | |
|-----------------------------|-----------------------------|
| 1) <code>tab2d[0]</code> | 3) <code>tab2d[2][3]</code> |
| 2) <code>tab2d[0][0]</code> | 4) <code>tab2d[1][3]</code> |

Un parcours de tableaux de tableaux se fait naturellement avec deux boucles `for` imbriquées.

On reprends note premier classique, la recherche d'un élément dans le tableau de tableaux.

```
1 def recherche2D(tab2d, val):
2     for t in tab2d : # on parcourt les tableaux internes
3         for elt in t : # on parcourt les valeurs de t
4             if val == elt :
5                 return True
6     return False
```

La version avec les positions.

```
1 def recherchepos2D(tab2d, val):
2     for i in range(len(tab2d)): # on parcourt les positions de tab2d
3         for j in range(len(tab2d[i])): # on parcourt les position du sous tableau
4             if tab2d[i][j] == val :
5                 return i,j # on renvoie le couple position
6     return None
```

Dans les deux cas, la complexité dans le pire des cas est en $O(nm)$ où n est le nombre de lignes et m le nombre de colonnes.

On peut bien sur étendre le principe à des tableaux de tableaux de tableaux, ou des tableaux contenant des tuples, etc.

IV RECHERCHE DES VALEURS LES PLUS PROCHES

Dans une situation où on a un ensemble de n points, on cherche une paire de points dont la distance est minimale. Cela peut s'adapter à des espaces métriques un peu bizarres, mais on se placera ici dans un ensemble de points du plan. Commençons par donner une distance dans le plan. Les points sont des couples de réels.

```
1 def distance( A,B):
2     xA,yA = A
3     xB,yB = B
4     return ((xB-xA)**2+(yB-yA)**2)**0.5
```

Un moyen simple mais lent, est de comparer toutes les paires possibles (sans doublons!). Il y a donc $\frac{n(n-1)}{2}$ paires à tester. L'ordre de grandeur est n^2 , on parle de complexité **quadratique** et on note $O(n^2)$.

```
1 def plus_proches(liste,distance):
2     n = len(liste)
3     paire = [0,1] # le premier couple de points
4     d = distance(liste[0],liste[1]) # la distance entre ces deux points
5     for i in range(n): # a compléter de manière optimisée
6         for j in range(i): # pareil
7             dij = distance(liste[i],liste[j])
8             if dij < d :
9                 paire = [i,j]
10                d =dij
11     return paire
```

Testons avec une liste de points `[(-2,1), (1,2), (2,1), (4,1), (4,3)]`.

V RECHERCHE DANS UN TEXTE

On dispose d'un mot (une chaîne de caractère) d'un texte (aussi) et on veut déterminer la présence du mot dans le texte. La difficulté ici est qu'on ne cherche pas une valeur mais une suite de valeurs dans une chaîne de caractères :

- il ne faut pas dépasser la chaîne
- il faut chercher la première lettre du mot, puis la seconde etc. sans sortir de la chaîne
- il faut tester la première lettre de la phrase, puis la seconde etc.

Cela devient essentiellement un exercice de rigueur. On souhaite renvoyer la position à laquelle le mot commence si il est présent. On va avoir besoin de deux indices :i pour se situer dans la phrase et j pour se situer dans le mot. Il suffit ensuite de parcourir judicieusement et faisant les tests.

Le mot est bien dans le texte si j arrive à la taille du mot!

```
1 def recherche_mot(mot, texte):
2     m = len(mot)
3     n = len(texte)
4     for i in range (n-m+1):
5         j=0
6         while j < m and mot[j]==texte[i+j]:
7             j+=1
8         if j== m:
9             return(i)
10    return(None)
```